

STRUCTURE OF A LEX SPECIFICATION

A lex program has three parts:

definition section

%%

rules section

%%

user defined functions

- The **definition section** usual contains, among other things, **literal blocks** and **pattern names**. We shall use the following specification in this section for our experiments :

%option noyywrap

In order for lex to work on greater than 1 input files, we are supposed to define a function yywrap(). This is not required in our experiments as we shall work with a single input file only.

- Literal blocks** provide a way to initialize variables used in the actions part. These are marked by the symbols %{ and %}. All the code within these markers are inserted verbatim in the initial part of the lex.yy.c file.
- The **pattern names** are shorthand for defining auxiliary patterns that are used to define complex patterns. These are inserted after the %{ and %} block and before the start of the %.
- The **rules section** contain a sequence of pattern – action pairs. The set of allowed patterns is fairly large and discussed later.
- A rule is a pattern - action pair. The set of pattern action pairs used in lex follows.

PATTERNS : The power of the generated lexical analyser depends on the descriptive power of patterns.

Pattern	Meaning	Example
x : x is a single character not in the lex meta-character set	The character x itself	A
\x : x is a single character in the lex meta-character set	The character x itself	*
“s” : s is a string of characters	The string s literally	“abc;”
[s] : s is a string of characters	Any single character occurring in s	[ABC]
[x-y] : x and y are characters	A shorthand for a character class defined by any character in the range from x to y	[0-9] any character from 0 to 9
[^s] : s is a string of characters	Any characters except those in strings	[^ \t\n] denotes the language consisting of all characters except

		a blank, a tab or a newline
. :	Any character except for a newline	Used to denote any single character that is not specified by other preceding regular expressions
p{m,n}: p is a pattern and m, n are integers	Number of m through n occurrences of the pattern p	X{1,4} denotes the strings {X, XX, XXX, XXXX}
(p) : p is a pattern	The same as p; the parentheses are used to define association	(ab)+ denotes one or more occurrences of ab
p1 p2 : p1 and p2 are patterns	The strings represented by p1 or p2	(ab) c+ denotes strings of the kind {ab, c, cc, ccc,...}
p1 p2 : p1 and p2 are patterns	The strings represented by p1 concatenated with the strings represented by p2	(ab)c+ denotes strings of the kind {abc, abcc, abccc,...}
p? : p is a pattern	Zero or one occurrences of the strings represented by p	(+ -)? denotes zero or 1 occurrence of + or -
p* : p is a pattern	Zero or more occurrences of the strings represented by p	a* denotes the strings {ε, a, aa, aaa, ...}
p+ : p is a pattern	One or more occurrences of the strings represented by p	a+ denotes the strings {a, aa, aaa, ...}
{n} : n is a pattern name specified in the definition section	The strings defined by the pattern corresponding to n	{digit} where digit 1 2 3 defines the strings corresponding to digit

There are other patterns as well and they will be introduced on a need to know basis.

Structure of the generated code in lex.yy.c

Global Definitions : C Definitions are inserted verbatim here Lex defined functions and data structures are also placed here
yylex(); User patterns and actions are used here
User defined functions main()